

PEC2 (Arquitectura de bases de datos)



TúNombre TúNombre TúApellido TúApellid

Arquitectura de bases de datos

04/12/2025

Índice

Índice.....	1
Pregunta 1 (2,5 puntos).....	2
a) Según el video, ¿por qué es necesario un sistema MapReduce y qué es exactamente?.....	2
b) ¿Cuáles son las operaciones que se pueden definir en un sistema MapReduce según el video?.....	2
c) ¿Qué aporta un esquema de procesamiento MapReduce cuando tenemos una base de datos distribuida?.....	3
d) ¿Se puede utilizar MapReduce como método de consulta y procesado de bases de datos? ¿Por qué?.....	3
Pregunta 2 (2,5 puntos).....	4
a. Explica brevemente cuáles son las posibles interferencias que se pueden dar en un SGDB. (0.5 puntos).....	4
b. Describe los mecanismos que ofrece Oracle para definir cómo se comportan las transacciones en relación con el nivel de aislamiento. Incluye los comandos necesarios para establecer dichos niveles. (0.5 puntos).....	4
c. Dada la siguiente tabla:.....	5
Crear la conexión T1 y crear en ella la tabla.....	6
Establecer el aislamiento en T1 y T2 en SERIALIZABLE.....	6
Realizar operaciones simultáneamente en T1 y T2.....	7
Conclusión:.....	8
d. ¿Qué pasaría en el apartado anterior si estableciéramos el nivel de aislamiento en READ COMMITTED? (0.5 puntos).....	9
Conclusión:.....	11
Pregunta 3 (2,5 puntos).....	11
a. Mirando los comandos del dietario, ¿en qué entorno de recuperación STEAL / NO STEAL sería posible que se produjera este dietario, sabiendo que se sigue una política NO FORCE? (0.5 puntos).....	12
b. ¿Qué posibles estrategias de checkpoint podemos aplicar y cuál se está utilizando? (0.5 puntos).....	12
c. En el supuesto de que nos encontramos en un entorno STEAL / NO FORCE, explica las acciones que debe realizar el sistema cuando se ejecuta la operación restart tras el fallo del sistema (buffer_and_transaction_list_reset, undo_to_last_CP_phase, undo_complementary_phase, redo_phase y enter_CP). (1.5 puntos).....	12
Pregunta 4 (2,5 puntos).....	13
Consulta reducida basada en fragmentos.....	14
Fase de reducción y optimización sintáctica.....	14
Aplicación de selecciones.....	14
Selección de fragmentos derivados.....	14
Eliminación de fragmentos irrelevantes.....	14
Consulta reducida.....	15
Justificación.....	15

Pregunta 1 (2,5 puntos)

Para realizar este ejercicio, es necesario mirar el vídeo “Bases de datos distribuidas: Framework “Map Reduce” que encontraréis a <https://vimeo.com/121674563>. Se pide que contestéis las siguientes preguntas (**Nota: la respuesta a este ejercicio no tiene que superar la extensión de dos páginas**):

a) Según el video, ¿por qué es necesario un sistema MapReduce y qué es exactamente?

Existen entornos con grandes volúmenes de datos e información distribuidos, entre ellos, por ejemplo, unos de gran importancia como los de **Big Data** donde el procesamiento de los datos de formas tradicionales es ineficiente, como ocurre con (casi) todas las tecnologías orientadas al desarrollo y diseño de software nacen **frameworks** que son librerías/entornos/herramientas dedicadas a mejorar y/o evolucionar la forma en la que trabajamos y procesamos, en este caso datos, **MapReduce** es uno de ellos, para ello, divide el procesamiento de datos en dos partes (que están en su nombre), “**Map**” y “**Reduce**”.

Cuando hablamos de **Map** nos referimos a la fase que transformas todos los datos de entrada en pares de estructura de datos (clave, valor) y cuando hablamos de **Reduce** nos referimos al proceso de agregar los valores por clave de los datos en, por ejemplo, los resultados finales, así permite, en paralelo, procesar conjuntos masivos de datos.

Esto permite gestionar grandes cantidades de información y procesarlas en paralelo lo que permite que la gestión de los mismos sea escalable y tenga un rendimiento, que por ejemplo, en sistemas tradicionales serían imposibles de manejar a menos que se tuviese más de un servidor encargado de ello, lo cual una vez más, sería ineficiente.

b) ¿Cuáles son las operaciones que se pueden definir en un sistema MapReduce según el video?

Un sistema MapReduce básico incluye las siguientes operaciones/fases:

- **Map (asignación):** Lee los datos de entrada por fragmentos y genera pares clave-valor intermedios. El mapper filtra, agrupa, ordena o transforma cada registro original en pares (clave, valor).
- **Sort (reordenación/agrupación):** Agrupa los pares intermedios por clave. El framework reordena las salidas de Map para que todas las tuplas con la misma clave lleguen juntas al mismo reductor. Esta fase de “barajado” suele incluir un ordenamiento por clave y prepara los datos para la agregación, en otras fuentes consultadas se llama alternativamente **Shuffle**.
- **Reduce (reducción):** Aplica una función de agregación o fusión sobre cada conjunto de valores asociados a una misma clave. Cada reducción procesa la lista de valores de una

clave determinada y produce el resultado final (por ejemplo, sumas, máximos, concatenaciones, etc.). El resultado de cada Reduce se almacena normalmente en el sistema de archivos distribuido o en la base de datos destino.

c) ¿Qué aporta un esquema de procesamiento MapReduce cuando tenemos una base de datos distribuida?

En las preguntas anteriores abordamos un concepto clave, el **procesamiento paralelo de conjuntos de datos**, esto funciona gracias a la ejecución en cluster, ejecutarse de esta forma permite aprovechar todos los nodos al mismo tiempo de almacenamiento,

MapReduce aporta procesamiento masivamente paralelo a bases de datos distribuidas. Al ejecutarse en un clúster, aprovecha todos los nodos de almacenamiento/cómputo simultáneamente. Gracias a su diseño, cada fragmento de datos se procesa en el nodo donde está guardado (lugar de almacenamiento de la base de datos), minimizando el movimiento de datos entre nodos y la latencia. Además, el framework de MapReduce (a través de componentes tipo Resource Manager y Node Manager) se encarga de asignar y orquestar las tareas en paralelo, así como de gestionar reintentos ante fallos, lo que garantiza tolerancia a fallos y alta fiabilidad. En conjunto, esto permite consultar y procesar grandes volúmenes de datos distribuidos de forma escalable: las tareas se paralelizan automáticamente, distribuyendo la carga de trabajo en todos los nodos de la base de datos distribuida y consolidando los resultados al final. De este modo, un esquema de procesamiento MapReduce enriquece a la base de datos distribuida con mayor rendimiento en análisis de datos masivos, aprovechando la capacidad de cálculo distribuido nativa del sistema.

d) ¿Se puede utilizar MapReduce como método de consulta y procesado de bases de datos? ¿Por qué?

MapReduce no es en sí mismo un lenguaje de consulta interactivo, sino un modelo de programación y procesamiento por lotes. En este sentido, no se emplea directamente como una interfaz SQL tradicional para realizar consultas ad-hoc en bases de datos relacionales. Sin embargo, algunos sistemas lo han integrado como mecanismo de procesamiento interno. Por ejemplo, ciertas bases de datos (como Netezza) permiten invocar tareas MapReduce mediante funciones definidas por el usuario desde sentencias SQL. En entornos Big Data, herramientas de capa superior (Hive, Pig) traducen consultas SQL o scripts en trabajos MapReduce subyacentes. En conclusión, sí es posible procesar datos de la base usando MapReduce, pero de forma indirecta: se necesitan desarrollar programas MapReduce o usar adaptaciones que transformen las consultas a jobs MapReduce. MapReduce fue concebido para cargas analíticas masivas (procesamiento batch) y no para consultas transaccionales interactivas, ya que cada consulta implicaría un costoso trabajo de planificación y lectura/escritura masiva de datos

Pregunta 2 (2,5 puntos)

a. Explica brevemente cuáles son las posibles interferencias que se pueden dar en un SGDB. (0.5 puntos)

Se me ocurre que podrían ocurrir interferencias en escenarios hipotéticos de transacciones simultáneas siendo probablemente el escenario más común en la realidad, esto ocurre por los siguientes motivos:

- **Lectura sucia:** Si, por ejemplo, una transacción lee datos modificados por otra transacción que aún no ha hecho commit y esta última transacción sin commit se aborta, los datos leídos nunca habrán sido válidos.
- **Lectura no repetible:** Otro escenario es cuando una transacción lee el mismo dato dos veces y obtiene valores distintos porque otra transacción lo ha modificado y confirmado entre ambas lecturas ocasionando un desfase en la información.
- **Lectura fantasma:** También puede ocurrir que una transacción "X" repite una consulta y aparecen o desaparecen filas debido a inserciones o borrados realizados por otra transacción que modifica el conjunto de datos.
- **Pérdida de actualizaciones:** Finalmente, si por ejemplo, dos transacciones actualizan el mismo dato y una sobrescribe los cambios de la otra sin tenerlos en cuenta entonces el propósito original de una de ellas es inconcluso.

b. Describe los mecanismos que ofrece Oracle para definir cómo se comportan las transacciones en relación con el nivel de aislamiento. Incluye los comandos necesarios para establecer dichos niveles. (0.5 puntos)

Los sistemas de gestión de bases basados y diseñados por y para Oracle implementan diferentes mecanismos que definen el comportamiento de transacciones, entre ellos, el control de concurrencia mediante **MVCC (Multiversion Concurrency Control)** y permite definir el aislamiento a nivel de sesión o transacción, los cuales son, por defecto, **READ COMMITTED** que, aunque evita lecturas sucias, permite todos los otros escenarios que pueden dar lugar a problemas y el modo **SERIALIZABLE**.

Este último garantiza un comportamiento equivalente a una ejecución secuencial de las transacciones, creando una fila **FIFO (First in, First out)** al menos desde mi punto de vista, donde, al seguir de forma secuencial las consultas se evita el solapamiento entre ellas y la lectura/escritura de datos en manipulación actual por otras transacciones, si bien podría ser una forma "tosca" de definirlo, es un ejemplo bastante ideal.

c. Dada la siguiente tabla:

```
CREATE TABLE BOOK
(
    IDBOOK INTEGER PRIMARY KEY,
    TITLE VARCHAR(50),
    AUTHOR VARCHAR(100)
);
INSERT INTO BOOK VALUES(1, 'Don Quixote', 'Miguel de Cervantes');
INSERT INTO BOOK VALUES (2, 'The House of Bernarda Alba','Federico Garcia Lorca');
INSERT INTO
```

Queremos ver cómo funciona el nivel **SERIALIZABLE** en Oracle. Se quiere ver qué comportamiento tienen al ejecutarse los siguientes comandos. En T1 queremos cambiar el nombre del primer libro por “Don Quixot”. En T2 queremos hacer lo mismo pero por “Don Quijote”.

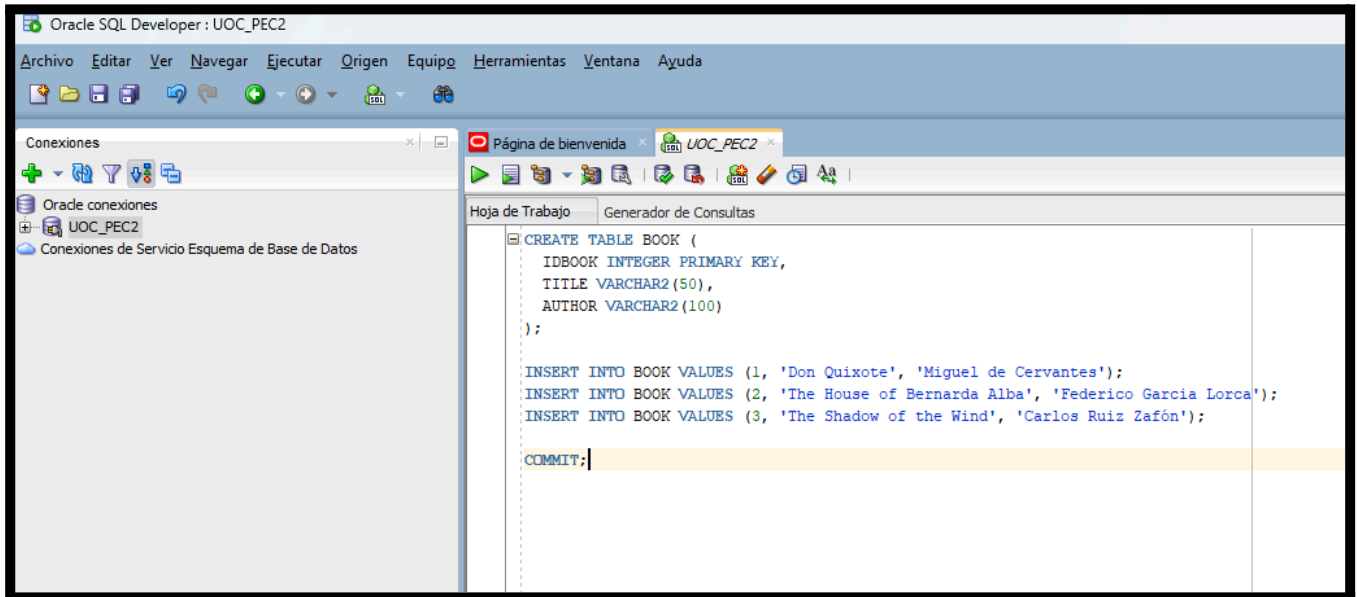
T1	T2
Establecer serialización	
	Establecer serialización
Select * from book;	
	Select * from book;
	update book set title = `Don Quijote` where IDBOOK = 1;
	commit;
update book set title = `Don Quixot` where IDBOOK = 1;	
commit;	

Para poder simular dos sesiones, es necesario abrir dos veces el Oracle SQL Developer (que nos permitirá simular dos transacciones diferentes). Indicad el comando a utilizar para establecer el aislamiento a **SERIALIZABLE** en las dos sesiones.

Mostrar el resultado de los diferentes comandos en las dos sesiones iniciadas adjuntando la captura de la ejecución. (1 punto)

Crear la conexión T1 y crear en ella la tabla

Con **Oracle XE** corriendo iniciamos el **Oracle SQL Developer** (uno de ellos), realizamos la conexión la primera vez y ejecutamos la consulta con la tabla dada en el enunciado.

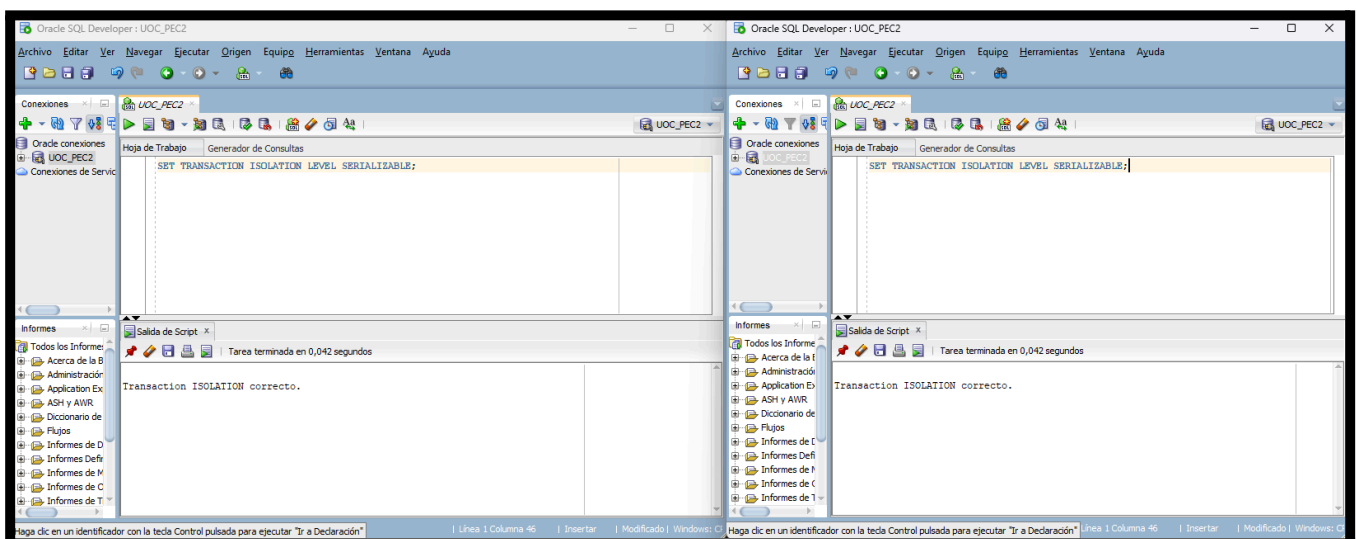


Establecer el aislamiento en T1 y T2 en SERIALIZABLE

Ahora que tenemos abiertas nuestras dos instancias de **ORACLE SQL DEVELOPER** debemos ejecutar, en ambas, la consulta/comando SQL siguiente:

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

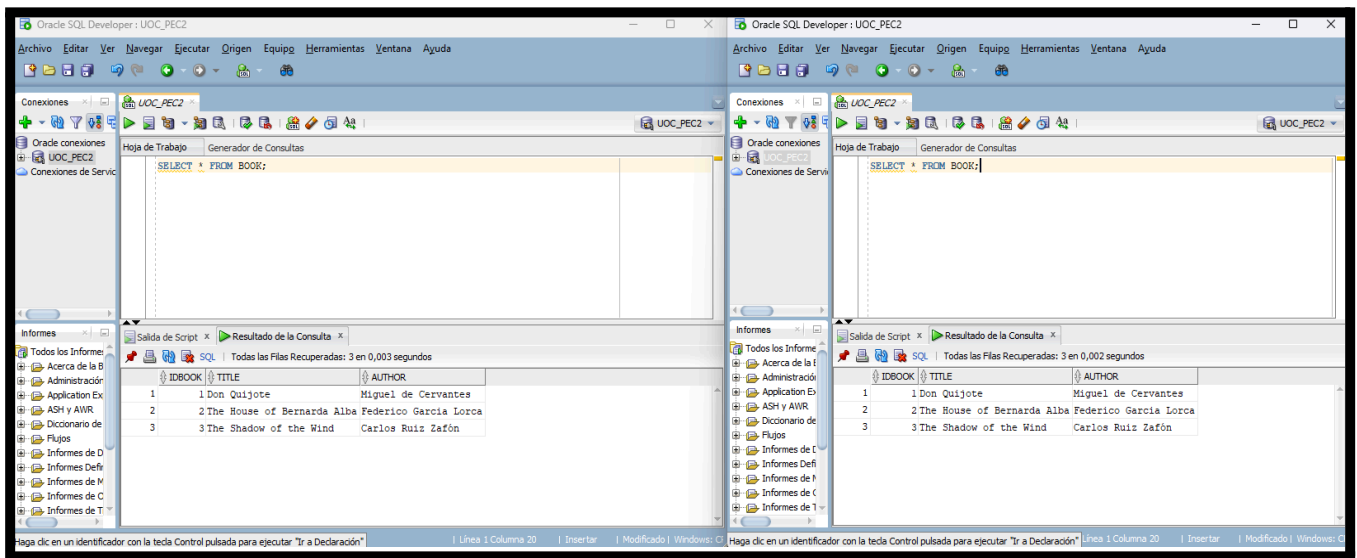
Para ello dejo adjuntas las capturas de pantalla de la ejecución del comandos en las dos ventanas de **ORACLE SQL DEVELOPER** aunque virtualmente parecen ser la misma.



Realizar operaciones simultáneamente en T1 y T2

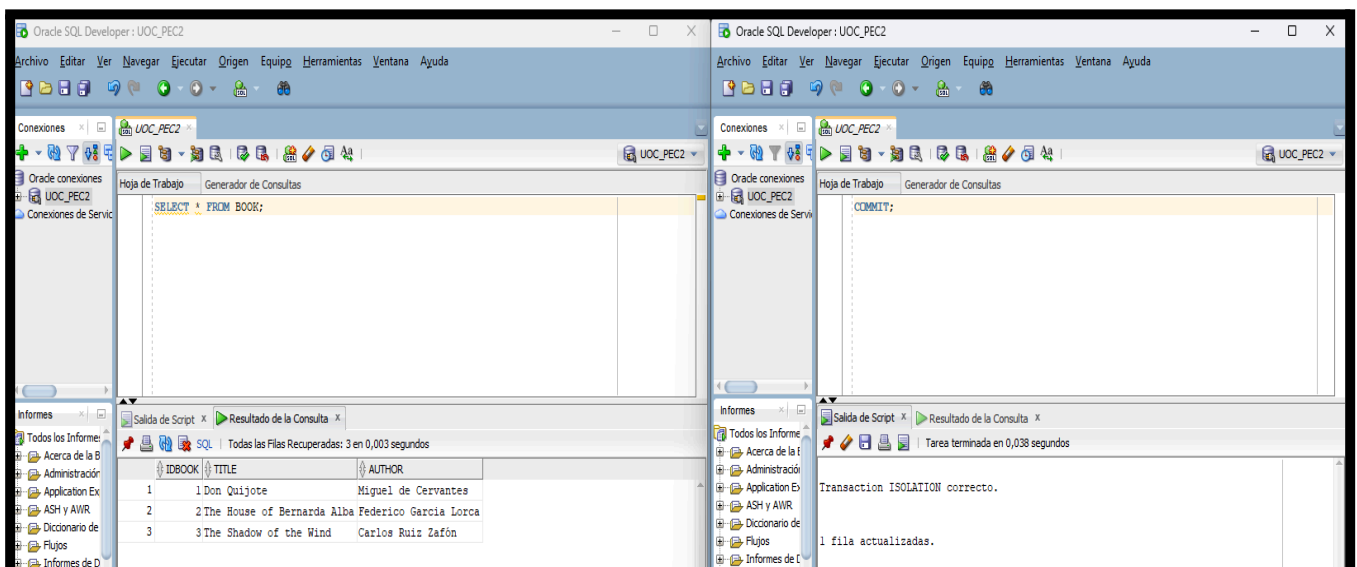
Para una mejor demostración las capturas de pantalla incluye de izquierda a derecha ambas sesiones de **ORACLE SQL DEVELOPER** para ver en todo momento desde la **UI** lo que sucede en ambas sesiones, empecemos por ejecutar la siguiente consulta en **PRIMERO EN T1** y **POSTERIORMENTE EN T2**:

SELECT * FROM BOOK;



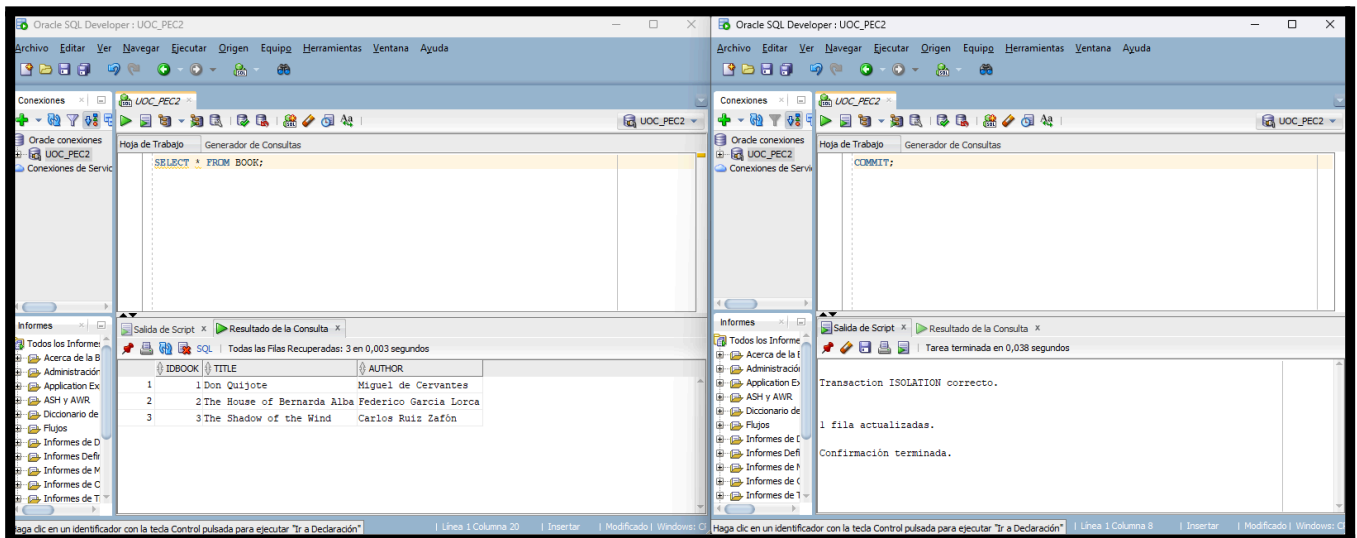
Ahora, según nuestra tabla guía desde **T2** debemos ejecutar la consulta de actualización del título del libro la cual es:

UPDATE BOOK SET TITLE = 'Don Quijote' WHERE IDBOOK = 1;



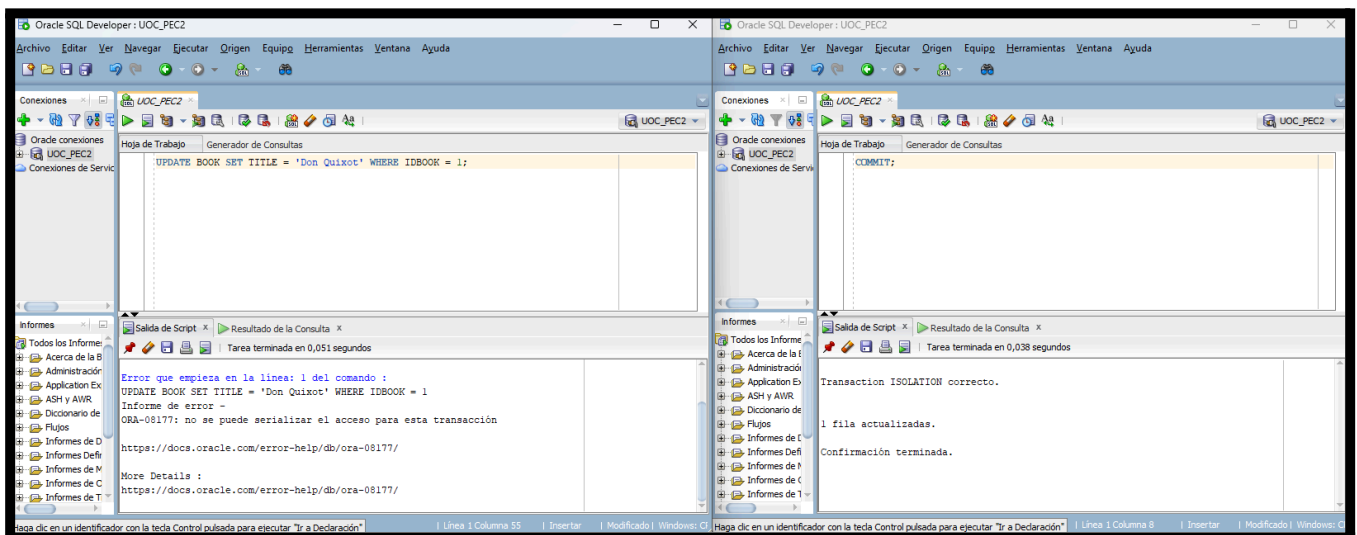
Ahora, desde la misma sesión **T2** tenemos que confirmar la transacción ejecutando el siguiente comando SQL:

COMMIT;



Ahora, según nuestra tabla guía intentaremos (ya confirmada la transacción en **T2**) modificar el título del libro con el siguiente comando SQL:

UPDATE BOOK SET TITLE = 'Don Quixot' WHERE IDBOOK = 1;



Conclusión:

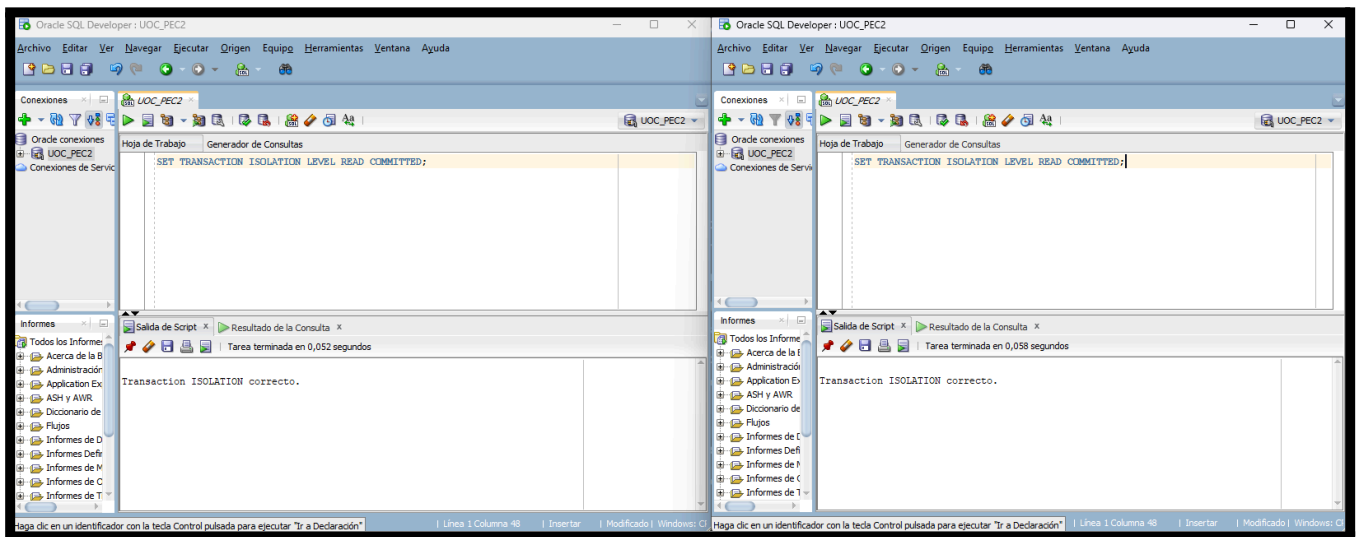
Lo que ocurre es que que **SERIALIZABLE** permite trabajar en simultáneo a varias “personas” con los mismos datos, cuando hacemos un select primero en **T1** este graba una “copia” local de los datos que usa como referencia, en ese breve lapso de tiempo **T2** modifica el

mismo conjunto de datos y **Oracle** cuando intenta modificar desde **T1** el mismo libre se da cuenta del desfase de la información marcando como invalida la consulta.

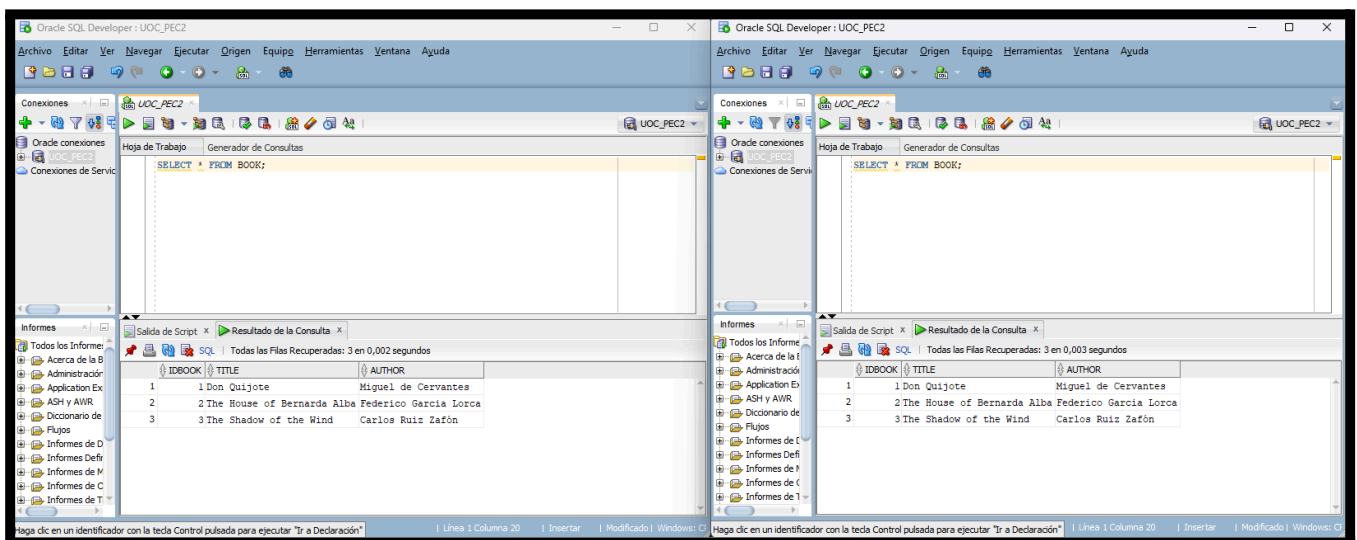
d. ¿Qué pasaría en el apartado anterior si estableciéramos el nivel de aislamiento en **READ COMMITTED**? (0.5 puntos)

La forma más rápida de saberlo es seguir la misma ejecución de los pasos anteriores pero iniciando ambas transacciones con el comando:

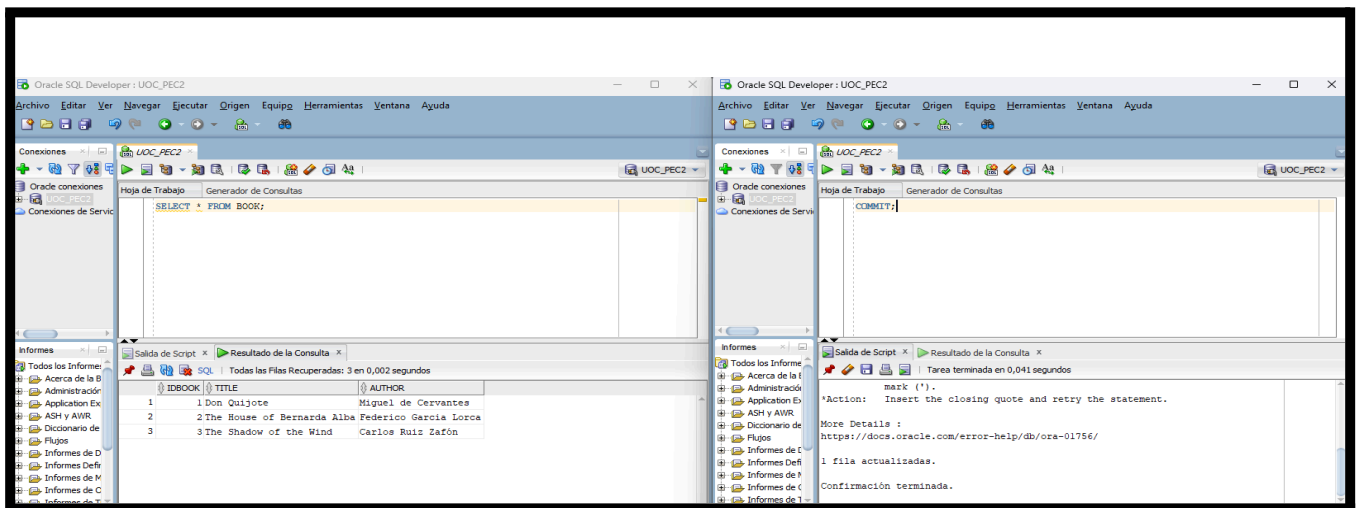
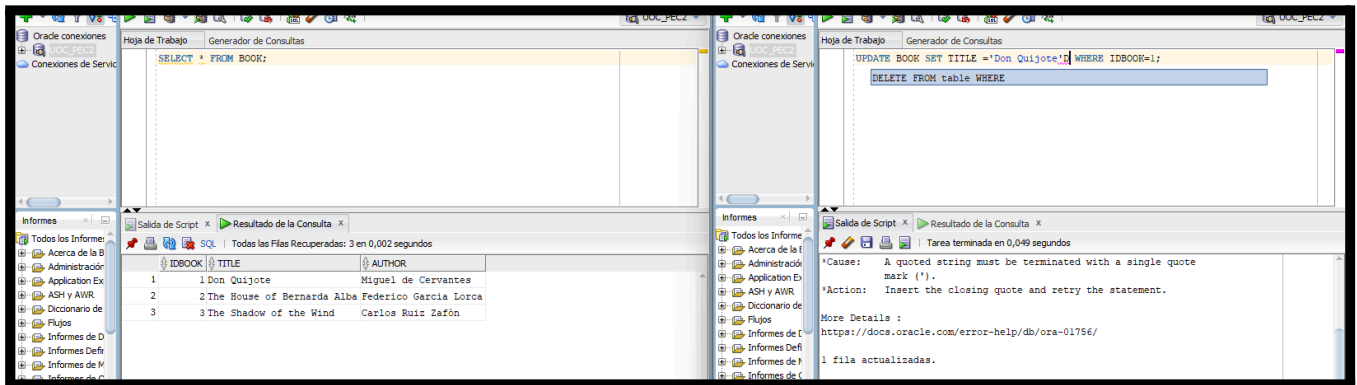
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;



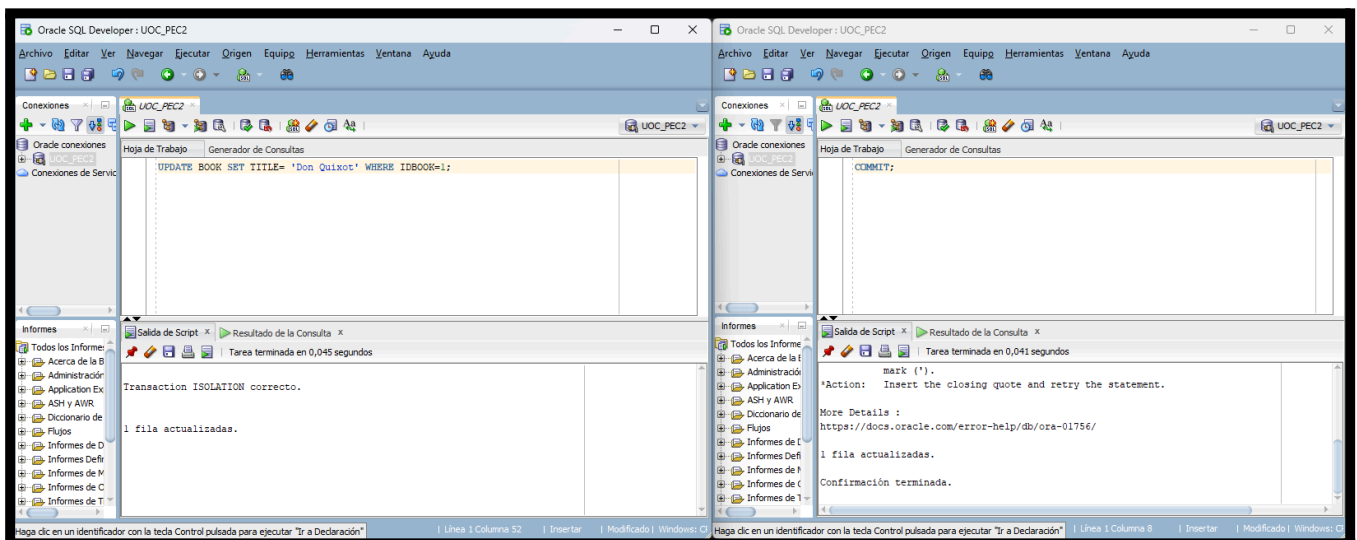
Ahora, primero en **T1** y luego en **T2** hacemos un **“SELECT * FROM BOOK;”** para que ambos graben los datos de la tabla de libros:

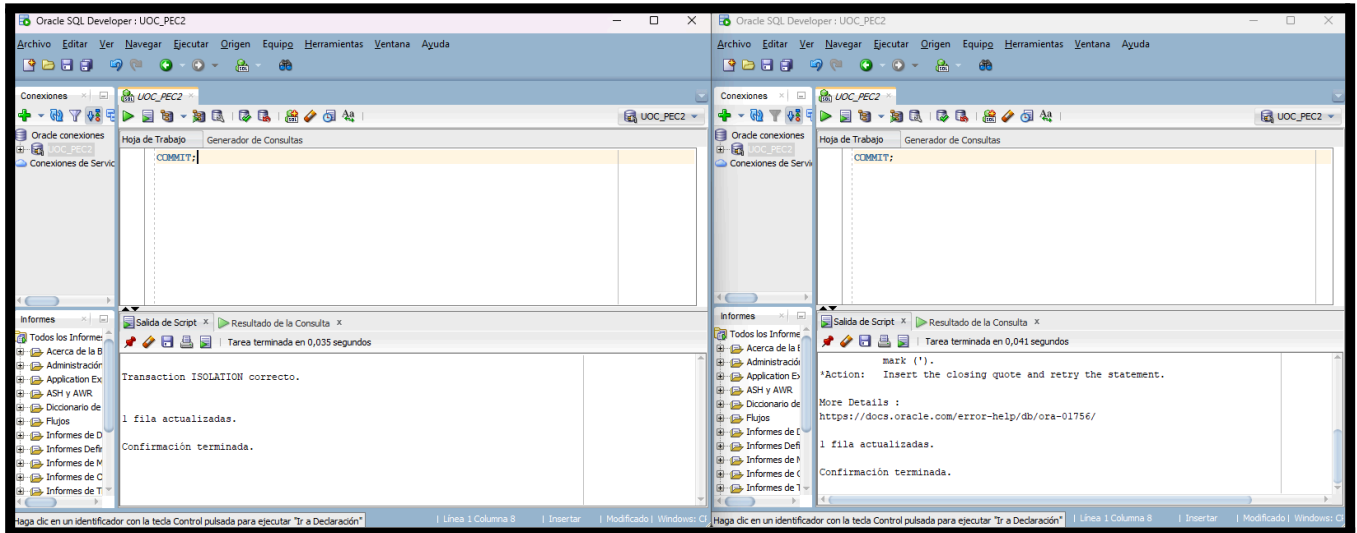


Ahora desde T2 tenemos que intentar actualizar el título del libro usando “**UPDATE BOOK SET TITLE='Don Quijote' WHERE IDBOOK=1;**” y hacer “**COMMIT**” de la actualización.



Finalmente, intentamos realizar los mismos cambios desde T1 usando el comando “**UPDATE BOOK SET TITLE='Don Quixot' WHERE IDBOOK=1;**” confirmando la transacción posteriormente con “**COMMIT;**”





Conclusión:

Aquí no ocurre ningún error porque **Oracle** lee los datos más recientes como referencia hasta de realizar, por ejemplo, modificaciones, por lo tanto, como **T2** ya ha finalizado su transacción, no impide que **T1** haga la suya aunque ambas involucren el mismo conjunto de datos y difieren de propósito y/o contenido.

Pregunta 3 (2,5 puntos)

Un SGBD está procesando transacciones y, en un momento dado, el contenido del dietario físico tiene las siguientes entradas:

1. 'u', T4, P7, 9, 3, nul
2. 'u', T3, P5, -2, 11, nul
3. 'u', T4, P2, 8, 16, 1
4. cp{ T4, T3 }
5. 'u', T2, P3, 1, 4, nul
6. 'u', T1, P6, 11, 5, nul
7. 'u', T5, P9, 5, 3, nul
8. cp{ T4, T2, T3, T1, T5 }
9. 'u' T2, P4, 16, 15, 5
10. 'u' T4, P7, 3, 1, 3
11. 'u' T1, P8, 2, 9, 6
12. 'c' T4
13. 'a' T2
14. SYSTEM FAILURE

Donde las entradas de update tienen el formato: 'u', transacción, página, imagen antes,

imagen después y apuntador al anterior registro de update de la transacción, el cual es nulo en su primera grabación de update de la transacción. **Se pide que respondáis a los siguientes apartados:**

- a. Mirando los comandos del dietario, ¿en qué entorno de recuperación **STEAL / NO STEAL** sería posible que se produjera este dietario, sabiendo que se sigue una política **NO FORCE**? (0.5 puntos).

Luego de examinar el contenido del dietario y sabiendo que la curva de dificultad radica en una política **NO FORCE**, el único entorno de recuperación posible es **STEAL / NO FORCE**.

Esto lo podemos deducir porque aparecen operaciones de actualización “(u)” de transacciones que no han finalizado (no hay commit visible) antes del fallo del sistema. En un entorno **STEAL**, el sistema permite escribir en disco páginas modificadas por transacciones no confirmadas, lo cual obliga a realizar operaciones de **UNDO** durante la recuperación. La política **NO FORCE** implica que, aunque una transacción se confirme, sus páginas modificadas no tienen por qué haberse volcado a disco en ese momento.

- b. ¿Qué posibles estrategias de checkpoint podemos aplicar y cuál se está utilizando? (0.5 puntos)

Desde mi punto de vista he llegado a la conclusión de que al menos una de las siguientes opciones es una posible estrategia de checkpoint aplicable:

- Checkpoint simple
- Checkpoint con lista de transacciones activas
- Checkpoint fuzzy

Ahora, podemos saber cual se utiliza, si si observamos el dietario aparecen entradas como “cp {T4, T3}” y “cp {T4, T2, T3, T1, T5}”, lo que indica que el sistema registra los checkpoints junto con el conjunto de transacciones activas en ese momento. Es por ese mismo motivo que la estrategia utilizada es un **checkpoint fuzzy** con lista de transacciones activas, ya que no se detiene la ejecución del sistema y se mantiene información suficiente para facilitar la recuperación tras un fallo.

- c. En el supuesto de que nos encontramos en un entorno **STEAL / NO FORCE**, explica las acciones que debe realizar el sistema cuando se ejecuta la operación restart tras el fallo del sistema (buffer_and_transaction_list_reset, undo_to_last_CP_phase, undo_complementary_phase, redo_phase y enter_CP). (1.5 puntos)

Suponiendo un entorno **STEAL / NO FORCE**, al producirse el fallo del sistema el SGBD ejecuta las siguientes fases durante la operación de restart:

- **buffer_and_transaction_list_reset**: Se limpia el buffer de memoria y se re-inicializa la

tabla de transacciones. A partir del último checkpoint se identifican las transacciones activas, confirmadas y abortadas.

- **undo_to_last_CP_phase:** El sistema retrocede hasta el último checkpoint registrado en el diario para limitar el número de operaciones a analizar durante la recuperación.
- **undo_complementary_phase:** Se deshacen (**UNDO**) todas las operaciones de las transacciones que no han confirmado antes del fallo, recorriendo el diario en orden inverso y utilizando los punteros a registros anteriores. Esto garantiza que no queden efectos de transacciones incompletas en la base de datos.
- **redo_phase:** Se rehacen (**REDO**) las operaciones de las transacciones que sí habían confirmado después del último checkpoint, asegurando que todos sus cambios persistentes se reflejan correctamente en la base de datos.
- **enter_CP:** Una vez finalizada la recuperación, el sistema genera un nuevo checkpoint y queda preparado para continuar con el procesamiento normal de transacciones.

Pregunta 4 (2,5 puntos)

La base de datos REREAD_BOOKS está formada por las relaciones siguientes, donde la clave primaria de cada relación está subrayada y las claves foráneas están en cursiva:

- **Authors** (idauthor, name, surname, birthdate, numberbooks, biography, nationality)
- **Books** (idbook, title, summary, *idtheme*, rate, writedyear) {*idtheme*: clave foránea a Themes}
- **Themes** (idtheme, theme, description)
- **WritedBy** (idrelation, *idbook*, *idauthor*, numberawards) {*idbook*: clave foránea a Books} {*idauthor*: clave foránea a Authors}

Sabemos también que esta base de datos está distribuida siguiendo la siguiente estrategia de fragmentación:

- **B1** = Books (writedyear > 1974)
- **B2** = Books (writedyear ≤ 1974)
- **A1** = Authors [idauthor, name, surname, birthdate, biography]
- **A2** = Authors [idauthor, nationality]
- **T1** = Themes (theme="historical")
- **T2** = Themes (theme <> "historical")
- **W1** = WritedBy ⋈ B1
- **W2** = WritedBy ⋈ B2

Dada la siguiente consulta global:

```
SELECT a.name, a.surname, b.title, t.description, w.numberawards
FROM Authors a, Books b, Themes t, WrittenBy w
WHERE t.theme = 'historical' and b.writedyear <= 1974 and b.idtheme = t.idtheme
and w.idbook = b.idbook and w.idauthor = a.idauthor
```

Encuentra la consulta reducida basada en fragmentos que ejecutará internamente el sistema gestor de bases de datos distribuido. Explica, basándote en los conceptos explicados en el M7 en referencia a la optimización sintáctica, los pasos principales que seguirá la fase de reducción, asumiendo que la estrategia de fragmentación es correcta.

Consulta reducida basada en fragmentos

El primer paso para esto es fragmentar la consulta dada en el enunciado, en mi caso he dado con la siguiente consulta basada en fragmentos:

```
SELECT a.name, a.surname, b.title, t.description, w.numberawards
FROM Authors a, Books b, Themes t, WrittenBy w
WHERE t.theme = 'historical'
AND b.writedyear <= 1974
AND b.idtheme = t.idtheme
AND w.idbook = b.idbook
AND w.idauthor = a.idauthor;
```

Fase de reducción y optimización sintáctica

Aplicación de selecciones

- La condición `t.theme = 'historical'` selecciona el fragmento **T1**.
- La condición `b.writedyear <= 1974` selecciona el fragmento **B2**.

Selección de fragmentos derivados

- A partir de **B2**, se selecciona el fragmento **W2**, definido como **WrittenBy** $\bowtie$ **B2**.
- De la relación **Authors**, se selecciona el fragmento **A1**, que contiene los atributos necesarios para la consulta.

Eliminación de fragmentos irrelevantes

Se descartan los fragmentos **T2**, **B1**, **W1** y **A2**, ya que no contribuyen a satisfacer las condiciones de la consulta ni aportan atributos requeridos en el resultado.

Consulta reducida

La consulta que ejecutará internamente el sistema gestor de bases de datos distribuido es (según mi criterio) la siguiente:

```
SELECT a.name, a.surname, b.title, t.description, w.numberawards  
FROM A1 a, B2 b, T1 t, W2 w  
WHERE b.idtheme = t.idtheme  
AND w.idbook = b.idbook  
AND w.idauthor = a.idauthor;
```

Justificación

La fase de reducción aplica las condiciones de selección lo antes posible, minimizando el volumen de datos transferidos entre nodos y ejecutando los joins únicamente entre fragmentos compatibles. Esto garantiza la corrección semántica de la consulta y mejora su eficiencia en un entorno distribuido.